
Welcome Offer Access to everything. Now up to 60% off. [Upgrade now](#)

Level Up Coding

This member-only story is on us.

★ Member-only story

Stop Juggling Microservices: The Spring Boot Monorepo Blueprint for AWS



Jitin Kayyala

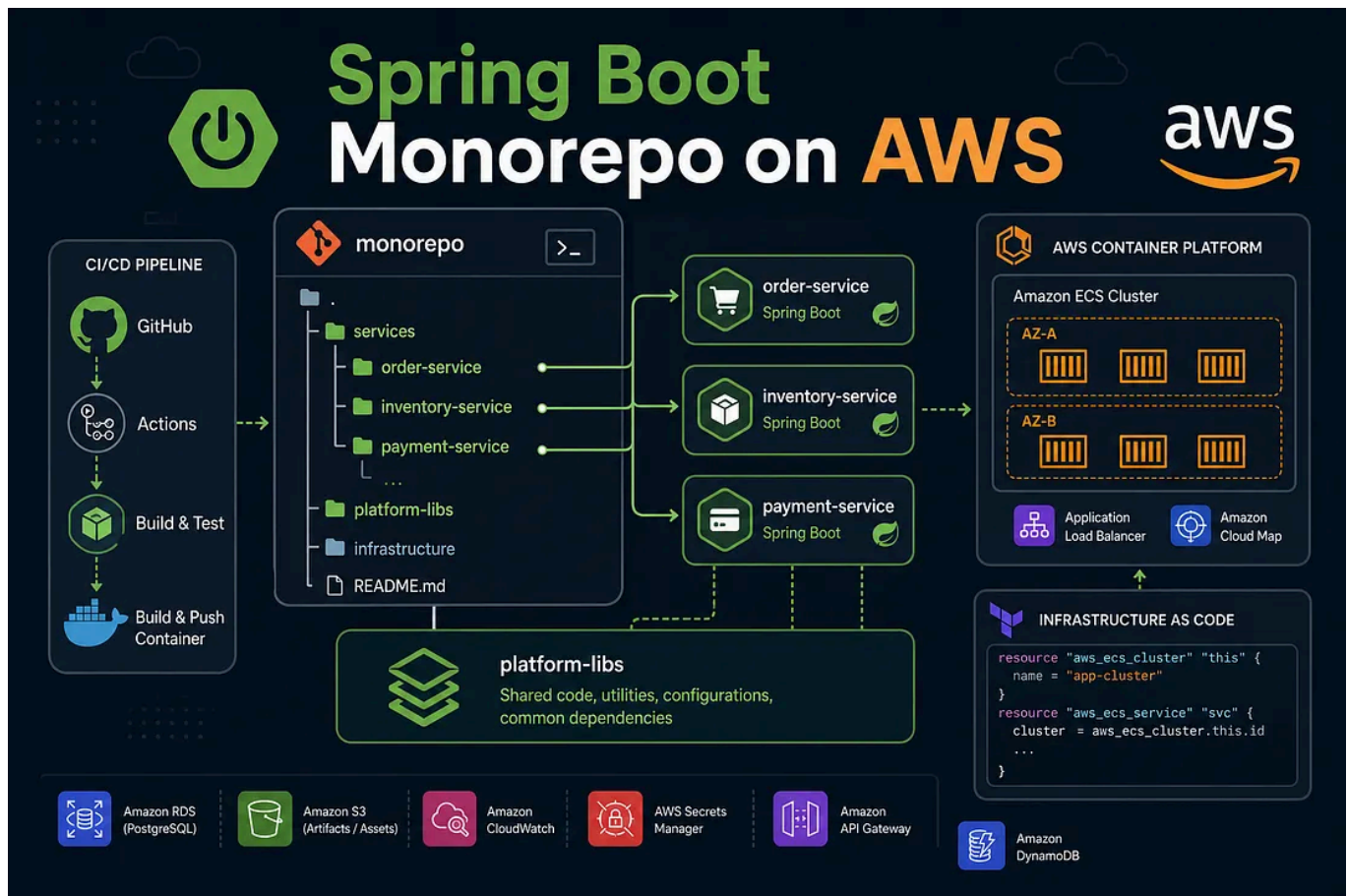
Following ▾

10 min read · May 6, 2026



142





Monorepo

If you have ever managed multiple Spring Boot micro-services scattered across different repositories, you will know the pain I am talking about.

Dependencies drift out of sync, shared code gets copy-pasted, and deploying a feature that touches multiple services becomes a coordination nightmare.

This was the pattern before I discovered the power of a well-structured monorepo. Not the “throw everything in one folder” approach that makes people cringe, but a *properly architected multi-module setup that gives you the best of both worlds: the cohesion of a monolith with the deployability of micro-services*.

The Case for Monorepos in Spring Boot

Let me walk you through a scenario that you could encounter. You are building an e-commerce platform with separate services for orders, inventory, payments, and notifications.

In the multi-repo world, you create four repositories. Then you need shared DTOs for inter-service communication. Where do they live? You create a fifth repo for common models. Now you need shared security utilities. Sixth repo. A month later, the order service is on version 1.2.0 of `common-models`, but payments is still on 1.0.5 because the team didn't notice the update. Good luck debugging the issue if it happens in production.

The monorepo approach changes the game.

All your services live in one repository, but they're independently deployable.

Shared code is just a module away, not a published artifact. Refactoring across services? One pull request. Version consistency? Guaranteed. CI/CD complexity? Actually reduced, not increased.

But here's the catch — structure matters enormously. A poorly organized monorepo is worse than multiple repos. You need deliberate boundaries, clear dependency rules, and a build system that understands your architecture.

The Anatomy of a Production-Ready Monorepo

The key insight is this: your repository structure should mirror your deployment architecture. If you're deploying to AWS ECS or EKS, each deployable service gets its own module. Shared logic lives in library modules that never deploy independently.

Here's the directory structure that's served me well across multiple projects:

```
my-platform/
├── platform-lib/
│   ├── common-domain/
│   ├── common-security/
│   ├── common-messaging/
│   └── common-persistence/
├── services/
│   ├── order-service/
│   │   ├── src/
│   │   ├── Dockerfile
│   │   └── build.gradle
│   ├── inventory-service/
│   ├── payment-service/
│   └── notification-service/
├── infrastructure/
│   ├── terraform/
│   │   ├── modules/
│   │   ├── environments/
│   │   └── service-configs/
│   ├── kubernetes/
│   │   └── manifests/
│   └── docker-compose.yml
├── gradle/
│   └── libs.versions.toml
├── settings.gradle
├── build.gradle
└── README.md
```

Shared libraries
Domain models, DTOs
Security utilities, JWT handlers
Kafka/SQS abstractions
JPA entities, base repositories
Deployable microservices

IaC and deployment configs

Local development
Centralized dependency versions
Root build configuration

This structure immediately communicates intent. Developers know where services live, where shared code belongs, and where infrastructure-as-code is maintained. More importantly, your CI/CD pipeline can make intelligent decisions about what to build and deploy based on what changed.

Gradle Configuration: The Foundation

Let's start with the build system. I'm using Gradle with the Groovy DSL here because it's more familiar to most Java developers and has a simpler syntax. The multi-project support is excellent either way.

settings.gradle - Project Structure Definition

```
rootProject.name = 'my-platform'

// Include all library modules
include 'platform-lib:common-domain'
include 'platform-lib:common-security'
include 'platform-lib:common-messaging'
include 'platform-lib:common-persistence'

// Include all service modules
include 'services:order-service'
include 'services:inventory-service'
include 'services:payment-service'
include 'services:notification-service'

// Set up logical project structure
project(':platform-lib:common-domain').projectDir = file('platform-lib/common-domain')
project(':platform-lib:common-security').projectDir = file('platform-lib/common-security')
project(':platform-lib:common-messaging').projectDir = file('platform-lib/common-messaging')
project(':platform-lib:common-persistence').projectDir = file('platform-lib/common-persistence')

project(':services:order-service').projectDir = file('services/order-service')
project(':services:inventory-service').projectDir = file('services/inventory-service')
project(':services:payment-service').projectDir = file('services/payment-service')
project(':services:notification-service').projectDir = file('services/notification-service')
```

gradle/libs.versions.toml - Centralized Version Management

This is where the magic happens. One file controls all dependency versions across your entire platform.

```
[versions]
spring-boot = "4.0.0"
spring-cloud = "2026.0.0"
java = "21"
aws-sdk = "2.25.0"
testcontainers = "1.19.0"

[libraries]
# Spring Boot
spring-boot-starter-web = { module = "org.springframework.boot:spring-boot-start
spring-boot-starter-data-jpa = { module = "org.springframework.boot:spring-boot-
spring-boot-starter-security = { module = "org.springframework.boot:spring-boot-
spring-boot-starter-actuator = { module = "org.springframework.boot:spring-boot-
spring-boot-starter-validation = { module = "org.springframework.boot:spring-boo

# Spring Cloud
spring-cloud-starter-aws = { module = "io.awspring.cloud:spring-cloud-aws-starte
spring-cloud-starter-aws-messaging = { module = "io.awspring.cloud:spring-cloud-

# AWS SDK
aws-sdk-s3 = { module = "software.amazon.awssdk:s3", version.ref = "aws-sdk" }
aws-sdk-dynamodb = { module = "software.amazon.awssdk:dynamodb", version.ref = "
aws-sdk-sqs = { module = "software.amazon.awssdk:sqs", version.ref = "aws-sdk" }

# Database
postgresql = { module = "org.postgresql:postgresql", version = "42.7.0" }
flyway-core = { module = "org.flywaydb:flyway-core", version = "10.0.0" }

# Testing
spring-boot-starter-test = { module = "org.springframework.boot:spring-boot-star
testcontainers-postgresql = { module = "org.testcontainers:postgresql", version.
testcontainers-localstack = { module = "org.testcontainers:localstack", version.

# Observability
micrometer-registry-cloudwatch = { module = "io.micrometer:micrometer-registry-c
opentelemetry-exporter-otlp = { module = "io.opentelemetry:opentelemetry-exporte

[plugins]
spring-boot = { id = "org.springframework.boot", version.ref = "spring-boot" }
spring-dependency-management = { id = "io.spring.dependency-management", version
```

Root build.gradle - Platform-Wide Configuration

```
plugins {  
    alias(libs.plugins.spring.boot) apply false  
    alias(libs.plugins.spring.dependency.management) apply false  
}  
  
subprojects {  
    group = 'com.mycompany.platform'  
    version = '1.0.0-SNAPSHOT'  
  
    repositories {  
        mavenCentral()  
    }  
  
    // Apply common configurations to all Java projects  
    pluginManager.withPlugin('java') {  
        sourceCompatibility = JavaVersion.VERSION_21  
        targetCompatibility = JavaVersion.VERSION_21  
  
        tasks.withType(Test) {  
            useJUnitPlatform()  
  
            // Useful for debugging tests in containers  
            testLogging {  
                events 'passed', 'skipped', 'failed'  
                exceptionFormat 'full'  
            }  
        }  
    }  
}
```

[Open in app](#)[Write](#)

The library modules are crucial. They prevent code duplication while maintaining clear boundaries. Here's how I structure them:

platform-lib/common-domain/build.gradle

```
plugins {  
    id 'java-library'
```

```
alias(libs.plugins.spring.boot) apply false
alias(libs.plugins.spring.dependency.management)

}

dependencies {
    // Just the essentials - no Spring Boot autoconfiguration
    implementation 'com.fasterxml.jackson.core:jackson-annotations'
    implementation 'jakarta.validation:jakarta.validation-api'

    // Lombok for reducing boilerplate
    compileOnly 'org.projectlombok:lombok'
    annotationProcessor 'org.projectlombok:lombok'
}

// This module doesn't produce an executable JAR
jar {
    enabled = true
}

bootJar {
    enabled = false
}
```

Sample Domain Model

```
package com.mycompany.platform.domain;

import jakarta.validation.constraints.*;
import lombok.*;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.UUID;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Order {
    private UUID id;

    @NotNull
    private UUID customerId;
```

```
@NotEmpty
private List<OrderItem> items;

@NotNull
@DecimalMin("0.01")
private BigDecimal totalAmount;

@NotNull
private OrderStatus status;

private Instant createdAt;
private Instant updatedAt;
}

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class OrderItem {
    private UUID productId;

    @NotBlank
    private String productName;

    @Min(1)
    private Integer quantity;

    @NotNull
    @DecimalMin("0.01")
    private BigDecimal unitPrice;
}

public enum OrderStatus {
    PENDING,
    CONFIRMED,
    PROCESSING,
    SHIPPED,
    DELIVERED,
    CANCELLED
}
```

platform-libs/common-security/build.gradle

```
plugins {  
    id 'java-library'  
    alias(libs.plugins.spring.boot) apply false  
    alias(libs.plugins.spring.dependency.management)  
}  
  
dependencies {  
    implementation libs.spring.boot.starter.security  
    implementation 'io.jsonwebtoken:jjwt-api:0.12.3'  
    runtimeOnly 'io.jsonwebtoken:jjwt-impl:0.12.3'  
    runtimeOnly 'io.jsonwebtoken:jjwt-jackson:0.12.3'  
  
    implementation project(':platform-lib:common-domain')  
}  
  
jar {  
    enabled = true  
}  
  
bootJar {  
    enabled = false  
}
```

JWT Security Configuration Example

```
package com.mycompany.platform.security;  
  
import io.jsonwebtoken.*;  
import io.jsonwebtoken.security.Keys;  
import org.springframework.beans.factory.annotation.Value;  
import org.springframework.stereotype.Component;  
  
import javax.crypto.SecretKey;  
import java.nio.charset.StandardCharsets;  
import java.time.Instant;  
import java.time.temporal.ChronoUnit;  
import java.util.Date;  
import java.util.UUID;  
  
@Component  
public class JwtTokenProvider {
```

```
private final SecretKey secretKey;
private final long expirationMillis;

public JwtTokenProvider(
    @Value("${jwt.secret}") String secret,
    @Value("${jwt.expiration-minutes:60}") long expirationMinutes) {
    this.secretKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    this.expirationMillis = expirationMinutes * 60 * 1000;
}

public String generateToken(UUID userId, String email, List<String> roles) {
    Instant now = Instant.now();
    Instant expiration = now.plus(expirationMillis, ChronoUnit.MILLIS);

    return Jwts.builder()
        .subject(userId.toString())
        .claim("email", email)
        .claim("roles", roles)
        .issuedAt(Date.from(now))
        .expiration(Date.from(expiration))
        .signWith(secretKey)
        .compact();
}

public Claims validateToken(String token) {
    try {
        return Jwts.parser()
            .verifyWith(secretKey)
            .build()
            .parseSignedClaims(token)
            .getPayload();
    } catch (JwtException | IllegalArgumentException e) {
        throw new InvalidTokenException("Invalid JWT token", e);
    }
}

public UUID extractUserId(String token) {
    Claims claims = validateToken(token);
    return UUID.fromString(claims.getSubject());
}
}
```

Service Modules: Independently Deployable

Now for the actual microservices. Each service is a complete Spring Boot application that can be built and deployed independently.

services/order-service/build.gradle

```
plugins {  
    id 'java'  
    alias(libs.plugins.spring.boot)  
    alias(libs.plugins.spring.dependency.management)  
}  
  
dependencies {  
    // Internal dependencies  
    implementation project(':platform-lib:common-domain')  
    implementation project(':platform-lib:common-security')  
    implementation project(':platform-lib:common-messaging')  
    implementation project(':platform-lib:common-persistence')  
  
    // Spring Boot starters  
    implementation libs.spring.boot.starter.web  
    implementation libs.spring.boot.starter.data.jpa  
    implementation libs.spring.boot.starter.security  
    implementation libs.spring.boot.starter.actuator  
    implementation libs.spring.boot.starter.validation  
  
    // AWS integration  
    implementation libs.spring.cloud.starter.aws  
    implementation libs.spring.cloud.starter.aws.messaging  
    implementation libs.aws.sdk.s3  
    implementation libs.aws.sdk.dynamodb  
  
    // Database  
    implementation libs.postgresql  
    implementation libs.flyway.core  
  
    // Observability  
    implementation libs.micrometer.registry.cloudwatch  
  
    // Testing  
    testImplementation libs.spring.boot.starter.test  
    testImplementation libs.testcontainers.postgresql  
    testImplementation libs.testcontainers.localstack  
}
```

```
// Enable bootJar for deployable services
bootJar {
    enabled = true
    archiveFileName = 'order-service.jar'
}

jar {
    enabled = false
}
```

Application Configuration — application.yml

```
spring:
  application:
    name: order-service

  datasource:
    url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:orders}
    username: ${DB_USERNAME:postgres}
    password: ${DB_PASSWORD:postgres}
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5
      connection-timeout: 30000

  jpa:
    hibernate:
      ddl-auto: validate
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
        format_sql: true
    show-sql: false

  flyway:
    enabled: true
    locations: classpath:db/migration
    baseline-on-migrate: true

# AWS Configuration
aws:
  region: ${AWS_REGION:us-east-1}
  sqs:
```

```
order-events-queue: ${ORDER_EVENTS_QUEUE_URL}
s3:
  invoice-bucket: ${INVOICE_BUCKET_NAME}

# Security
jwt:
  secret: ${JWT_SECRET}
  expiration-minutes: 60

# Actuator endpoints
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: always
  metrics:
    export:
      cloudwatch:
        namespace: MyPlatform/OrderService
        enabled: true
        step: 1m

server:
  port: ${PORT:8080}
  shutdown: graceful

# Logging
logging:
  level:
    com.mycompany.platform: ${LOG_LEVEL:INFO}
    org.springframework.web: INFO
    org.hibernate.SQL: DEBUG
  pattern:
    console: "%d{yyyy-MM-dd HH:mm:ss} - %msg%n"
```

Sample REST Controller

```
package com.mycompany.platform.order.controller;

import com.mycompany.platform.domain.Order;
```

```
import com.mycompany.platform.order.service.OrderService;
import com.mycompany.platform.security.JwtTokenProvider;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.UUID;

@Slf4j
@RestController
@RequestMapping("/api/v1/orders")
@RequiredArgsConstructor
public class OrderController {

    private final OrderService orderService;
    private final JwtTokenProvider jwtTokenProvider;

    @PostMapping
    @PreAuthorize("hasRole('CUSTOMER')")
    public ResponseEntity<Order> createOrder(
        @Valid @RequestBody CreateOrderRequest request,
        @RequestHeader("Authorization") String authHeader) {

        UUID userId = extractUserIdFromToken(authHeader);
        log.info("Creating order for user: {}", userId);

        Order order = orderService.createOrder(userId, request);
        return ResponseEntity.status(HttpStatus.CREATED).body(order);
    }

    @GetMapping("/{orderId}")
    @PreAuthorize("hasAnyRole('CUSTOMER', 'ADMIN')")
    public ResponseEntity<Order> getOrder(@PathVariable UUID orderId) {
        return orderService.findById(orderId)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @GetMapping
    @PreAuthorize("hasRole('CUSTOMER')")
    public ResponseEntity<List<Order>> getUserOrders(
        @RequestHeader("Authorization") String authHeader,
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size) {
```

```
UUID userId = extractUserIdFromToken(authHeader);
List<Order> orders = orderService.findByUserId(userId, page, size);
return ResponseEntity.ok(orders);
}

@PatchMapping("/{orderId}/status")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<Order> updateOrderStatus(
    @PathVariable UUID orderId,
    @Valid @RequestBody UpdateOrderStatusRequest request) {

    Order updated = orderService.updateStatus(orderId, request.getStatus());
    return ResponseEntity.ok(updated);
}

private UUID extractUserIdFromToken(String authHeader) {
    String token = authHeader.replace("Bearer ", "");
    return jwtTokenProvider.extractUserId(token);
}
}
```

Docker Configuration for AWS Deployment

Each service gets its own optimized Dockerfile. Here's a production-ready multi-stage build:

services/order-service/Dockerfile

```
# Stage 1: Build
FROM eclipse-temurin:21-jdk-alpine AS builder

WORKDIR /workspace

# Copy gradle wrapper and build files
COPY gradle gradle
COPY gradlew .
COPY settings.gradle .
COPY build.gradle .
COPY gradle/libs.versions.toml gradle/
```

```
# Copy all modules (needed for dependencies)
COPY platform-libs platform-libs
COPY services/order-service services/order-service

# Build the specific service
RUN ./gradlew :services:order-service:bootJar --no-daemon

# Extract layers for better Docker caching
RUN mkdir -p services/order-service/build/extracted && \
    java -Djarmode=layertools \
        -jar services/order-service/build/libs/order-service.jar \
        extract --destination services/order-service/build/extracted

# Stage 2: Runtime
FROM eclipse-temurin:21-jre-alpine

# Add non-root user for security
RUN addgroup -g 1001 spring && adduser -u 1001 -G spring -s /bin/sh -D spring

WORKDIR /app

# Copy extracted layers
COPY --from=builder --chown=spring:spring /workspace/services/order-service/build/extracted services/order-service/build/extracted
COPY --from=builder --chown=spring:spring /workspace/services/order-service/build/libs/order-service.jar services/order-service/build/libs/order-service.jar
COPY --from=builder --chown=spring:spring /workspace/services/order-service/build/resources/main services/order-service/build/resources/main
COPY --from=builder --chown=spring:spring /workspace/services/order-service/build/resources/test services/order-service/build/resources/test

USER spring

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=40s --retries=3 \
    CMD wget --quiet --tries=1 --spider http://localhost:8080/actuator/health || exit 1

EXPOSE 8080

ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

Local Development Setup

A good monorepo needs excellent local dev experience. Docker Compose is perfect:

docker-compose.yml

```
version: '3.8'

services:
  postgres-orders:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: orders
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres-orders-data:/var/lib/postgresql/data

  postgres-inventory:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: inventory
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5433:5432"
    volumes:
      - postgres-inventory-data:/var/lib/postgresql/data

  localstack:
    image: localstack/localstack:latest
    environment:
      SERVICES: s3,sqs,dynamodb,secretsmanager
      DEBUG: 1
      AWS_DEFAULT_REGION: us-east-1
    ports:
      - "4566:4566"
    volumes:
      - localstack-data:/var/lib/localstack
      - ./infrastructure/localstack-init:/etc/localstack/init/ready.d

  order-service:
    build:
      context: .
      dockerfile: services/order-service/Dockerfile
    environment:
      DB_HOST: postgres-orders
      DB_PORT: 5432
      DB_NAME: orders
      DB_USERNAME: postgres
      DB_PASSWORD: postgres
      AWS_REGION: us-east-1
```

```
AWS_ENDPOINT_OVERRIDE: http://localstack:4566
ORDER_EVENTS_QUEUE_URL: http://localstack:4566/000000000000/order-events
INVOICE_BUCKET_NAME: invoices-local
JWT_SECRET: local-dev-secret-key-change-in-production
LOG_LEVEL: DEBUG

ports:
  - "8080:8080"

depends_on:
  - postgres-orders
  - localstack

inventory-service:
  build:
    context: .
    dockerfile: services/inventory-service/Dockerfile
  environment:
    DB_HOST: postgres-inventory
    DB_PORT: 5432
    DB_NAME: inventory
    DB_USERNAME: postgres
    DB_PASSWORD: postgres
    AWS_REGION: us-east-1
    AWS_ENDPOINT_OVERRIDE: http://localstack:4566
    LOG_LEVEL: DEBUG
  ports:
    - "8081:8080"
  depends_on:
    - postgres-inventory
    - localstack

volumes:
  postgres-orders-data:
  postgres-inventory-data:
  localstack-data:
```

Testing Strategy

One massive benefit of monorepos: cross-module integration tests are trivial.

Integration Test Example

```

package com.mycompany.platform.order;

import com.mycompany.platform.domain.Order;
import com.mycompany.platform.domain.OrderStatus;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.web.client.TestRestTemplate;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.containers.localstack.LocalStackContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;
import org.testcontainers.utility.DockerImageName;

import static org.assertj.core.api.Assertions.assertThat;
import static org.testcontainers.containers.localstack.LocalStackContainer.Servi

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@Testcontainers
class OrderServiceIntegrationTest {

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres
        .withDatabaseName("orders_test")
        .withUsername("test")
        .withPassword("test");

    @Container
    static LocalStackContainer localstack = new LocalStackContainer(
        DockerImageName.parse("localstack/localstack:latest"))
        .withServices(SQS);

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
        registry.add("aws.endpoint-override",
            () -> localstack.getEndpointOverride(SQS).toString());
    }

    @Autowired
    private TestRestTemplate restTemplate;

```

```
@Test
void shouldCreateOrderSuccessfully() {
    CreateOrderRequest request = CreateOrderRequest.builder()
        .customerId(UUID.randomUUID())
        .items(List.of(
            OrderItemRequest.builder()
                .productId(UUID.randomUUID())
                .productName("Test Product")
                .quantity(2)
                .unitPrice(new BigDecimal("29.99"))
                .build()
        ))
        .build();

    ResponseEntity<Order> response = restTemplate
        .withBasicAuth("test-user", "test-password")
        .postForEntity("/api/v1/orders", request, Order.class);

    assertThat(response.getStatusCode()).isEqualTo(HttpStatus.CREATED);
    assertThat(response.getBody()).isNotNull();
    assertThat(response.getBody().getStatus()).isEqualTo(OrderStatus.PENDING);
    assertThat(response.getBody().getTotalAmount())
        .isEqualToByComparingTo(new BigDecimal("59.98"));
}
```

Some issues and Solutions

Let me be honest about the challenges I have faced:

Problem 1: Build times in CI

When you have 10+ services, `./gradlew build` takes forever. Solution: Use Gradle's `--parallel` flag and configure build caching properly. In GitHub Actions, cache the `.gradle` directory and use `actions/cache`.

Problem 2: Merge conflicts in shared files

`gradle/libs.versions.toml` becomes a hotspot. Solution: Keep dependency updates in small, frequent PRs.

Problem 3: Service boundaries

Teams start coupling services through shared libraries. Solution: Establish clear rules — shared libraries are for DTOs and utilities, not business logic. Use architecture tests with ArchUnit to enforce this.

Problem 4: Database migrations across environments

With multiple services, Flyway migrations can conflict. Solution: Namespace your migrations (`V001__orders_`, `V001__inventory_`) and use separate schemas or databases per service.

Key Takeaways

A well-structured Spring Boot monorepo for AWS deployments gives you:

1. **Single source of truth** for dependencies and versions
2. **Atomic refactoring** across service boundaries
3. **Simplified CI/CD** with intelligent change detection
4. **Better developer experience** with consistent tooling
5. **Independent deployability** of services despite code co-location

Programming ✓

Technology ✓

Monorepo +

Java +

AWS +



Published in Level Up Coding

342K followers · Last published 1 day ago

Following

Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev



Written by Jitin Kayyala

299 followers · 50 following

Following ▾

